Jenkins + SonarQube Classroom Setup Guide

Four-team classroom model • Community Edition / Community Build

This guide summarizes a practical classroom architecture for validating Jenkins and SonarQube across four student teams. The instructor centrally manages SonarQube projects, analysis tokens, quality profiles, and quality gates; each team configures Jenkins to submit its own analysis.

1. Recommended Classroom Architecture

Onne SonarQube project has been created per team. Each team has a project-specific analysis token and needs to store it as a Jenkins Secret Text credential. The token authorizes Jenkins to submit analysis; it does not select programming-language rules or define the quality gate.

Team	SonarQube Project Key	Token	Jenkins Credential ID
Team 1	team1-app	Team 1 project-analysis token	sonarqube-token
Team 2	team2-app	Team 2 project-analysis token	sonarqube-token
Team 3	team3-app	Team 3 project-analysis token	sonarqube-token
Team 4	team4-app	Team 4 project-analysis token	sonarqube-token

Suggested flow:

- GitHub repository → Jenkins checkout/build/test
- Jenkins → SonarQube analysis using the team's project key and token
- SonarQube → applies language-specific Quality Profiles
- SonarQube → evaluates the common Classroom Quality Gate
- Jenkins → continues on PASS or stops on FAIL

2. Tokens: What They Do

A SonarQube analysis token is an authentication credential. It proves that Jenkins is authorized to submit analysis to SonarQube. For classroom isolation, use one project-analysis token per team/project.

> ***You do NOT need separate tokens for Java, Python, TypeScript/Angular, SQL, or other languages.***

If Team 1 has one SonarQube project containing Java, Python, and TypeScript source, the same Team 1 project-analysis token can be used for that project. Separate tokens become relevant only if you create separate SonarQube projects (for example, team1-frontend and team1-backend).

3. Students Setup in SonarQube

Import Quality Profiles

Quality Profiles define which code-analysis rules run. Profiles are language-specific, so you can maintain classroom profiles for Java, Python, JavaScript/TypeScript, and other supported languages. Please find your profiles in the texoma-sonarqube-config up under the Extras folder in the Dailies (Section 8)

Students will be responsible for setting up their SonarQube credentials in Jenkins (see section 6)

Students will be responsible for setting up their team Jenkins with the appropriate SonarQube Analysis Steps (see section 6)

4. Quality Profiles vs. Quality Gates

Concept	Question It Answers	Instructor Controls
Quality Profile	Which coding rules should be checked?	Enabled/disabled rules and language-specific profiles
Quality Gate	Is the analyzed code acceptable?	PASS/FAIL conditions and thresholds
Analysis Token	Is Jenkins authorized to submit analysis?	Project/user authentication and permissions

5. Example Classroom Quality Gate

Choose thresholds appropriate to the course objectives and SonarQube version. A simple teaching gate might include:

- Coverage on new code: at least 80%
- Duplicated lines on new code: no more than 3%
- Reliability/security conditions: no unacceptable new-code issues

The exact metrics available and their names can vary by SonarQube version. Pinning the same SonarQube version for instructor and student environments helps keep results consistent.

6. Student Jenkins Configuration

Students should store the token in Jenkins Credentials rather than in Git, a Jenkinsfile, Docker image, application configuration, or shell script. Team Tokens can be found at the bottom of this page.

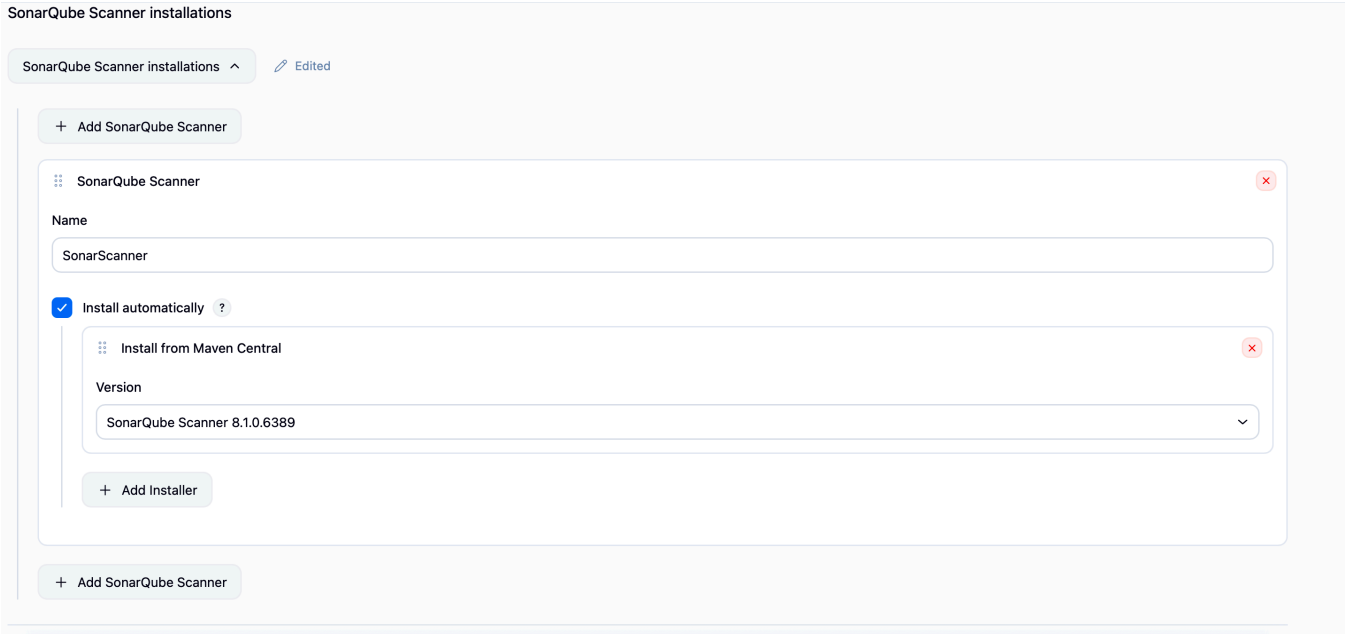
Students should install the following Plugins into their Jenkins Configuration:

- SonarQube Scanner

Referenced Documentation Setup and Usage

1. https://docs.sonarsource.com/sonarqube-community-build/analyzing-source-code/ci-integration/jenkins-integration/global-setup

2. In Jenkins/Tools install a SonarQube Scanner and name it 'SonarScanner'.
Manage Jenkins → Tools → SonarQube Scanner installations



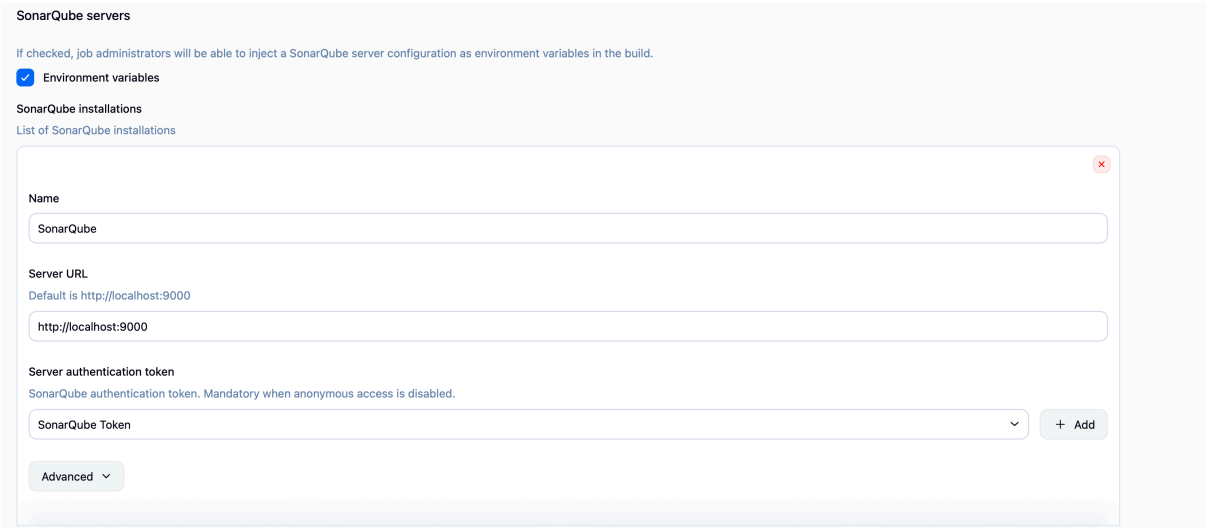
3. Configure the Sonar Qube Plugin to point to linux server where SonarQube is running:

Manage Jenkins

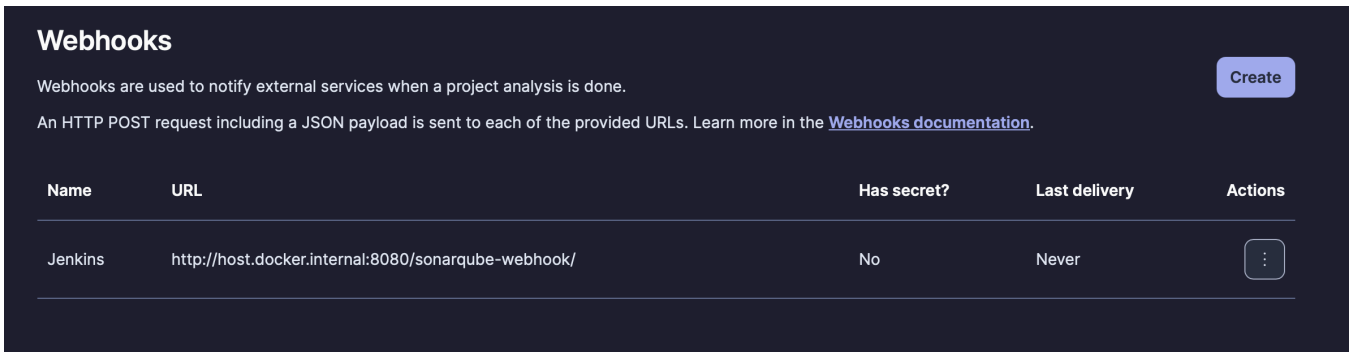
→ Configure System

→ From the Sonar Qube Section select 'Add SonarQube'

→ Update with IP address for where you are running sonar qub using credentials you established earlier.



4. Configure a WebHook in SonarQube so Jenkins can communicate back to SonarQube. You can add a WebHook by going to the 'Administration' menu for Sonar Qube. Use a URL suitable for your SonarQube installation. In this example SonarQube is running as a docker container as demonstrated in class.



5. Update your sonar-project.properties file to point to your coverage reports.

Make a copy of the sonar-project.properties file. It should be placed in the same directory as your jenkins file.

Java Example
sonar.coverage.jacoco.xmlReportPaths=\
java-service/target/site/jacoco/jacoco.xml

Important: When Configuring the Sonar Qube Plugin use SonarQube as the configuration name in Jenkins setup.

This will simplify your Jenkins update to the following:

```
stage('SonarQube Analysis') {
        steps {
                script {
                        def scannerHome = tool 'SonarScanner'
                        withSonarQubeEnv('SonarQube') {
                                sh """
                                    echo "${scannerHome}"
                                    ${scannerHome}/bin/sonar-scanner --version
                                    ${scannerHome}/bin/sonar-scanner
                                """
                        }
                }
        }
}
```

Note: SonarQube analyzes the source and imports the coverage/test reports produced by Jacoco, PyTest and other tools across your repository. One SonarQube analysis does not mean one build/test process. The student still needs the to build/test Java, Angular and Python appropriately with proper unit testing as before.

7. What the Instructor Will Validate

- Jenkins successfully checks out and builds the team's GitHub project.
- Automated tests run before or as part of analysis.
- Jenkins authenticates to SonarQube without exposing the token.
- Analysis lands in the correct team's SonarQube project.
- The expected language Quality Profiles/rules are applied.
- The common Classroom Quality Gate is evaluated.
- A failed gate prevents the pipeline from proceeding when that is the assignment requirement.

8. Sharing the Same Rules and Gates for Local SonarQube

Students can run local SonarQube environments that closely match the instructor server. For reproducibility, use the same SonarQube version and distribute the classroom configuration.

Quality Profiles

Custom Quality Profiles can be backed up/exported and restored in compatible SonarQube installations. This is useful for distributing your Java, Python, and TypeScript rule selections.

Quality Gates

For repeatable setup, treat the gate as configuration and automate its creation with SonarQube's Web API rather than relying on students to reproduce the thresholds manually.

Suggested configuration repository:

```
sonarqube-config/
├── README.md
├── quality-profiles/
│   ├── java.xml
│   ├── python.xml
│   └── typescript.xml
└── docker-compose.yml
```

A student can start the pinned SonarQube container, run the configuration script, restore the classroom profiles, and create the same quality gate. The instructor SonarQube server remains authoritative for grading.

9. Quality Control Gates

Add the following quality gates to your sonar-qube installation. These will be the same gates applied to your Jenkins run when reviewed at the end of each sprint.



Conditions on Overall Code

Metric	Operator	Value
Duplicated Lines (%)	is greater than	3.0%
High Severity Issues	is greater than	1
Medium Severity Issues	is greater than	5
Security Issue Severity on Overall Code	is greater than or equal to	Minor
Security Rating	is worse than	A
Skipped Unit Tests	is greater than	1
Technical Debt Ratio	is greater than	5.0%



Conditions on New Code

Metric	Operator	Value
Issues	is greater than	0
Security Hotspots Reviewed	is less than	100%
Coverage	is less than	95.0%
Duplicated Lines (%)	is greater than	2.0%
High Severity Issues	is greater than	1
Medium Severity Issues	is greater than	3
Security Rating	is worse than	A
Technical Debt Ratio	is greater than	3.0%

10. Reference Documentation

SonarQube Community Build documentation topics to consult:

- Managing authentication tokens
- Creating and administering projects
- Jenkins integration and global setup
- Adding analysis to a Jenkins pipeline
- Pausing a Jenkins pipeline for a Quality Gate
- Webhooks
- Quality Profiles and Quality Gates

11. Team Tokens

These tokens will be used by the instructor when he runs the Jenkins for each team. He will change his/her server config for SonarQube to point to the appropriate token for each team. The teams can have their own tokens for their projects locally, while the instructor uses these tokens for each project.

DuaLeap: sqp_b1b0bdb8ac15973c101e758e15eaf941b409aaef

LilLeap: sqp_45c94130ca2c39972eb4a52b0b383ed1f1f13cd5

LeMarketJames: sqp_473353d0d46106dcedf0647fca65ed9130cc18f2

LeapLaughLove: sqp_3a89d3a4f774e7efaa5fee2874270d86e0039db1

12. Team Project Names and Keys

Each teach will be assigned a team name and key. Each team will need to update their project name and key to align with the ones used by the instructor.

Property	Purpose	Must match SonarQube?
`sonar.projectKey`	Unique identifier SonarQube uses to locate the project	**Yes — this is critical**
`sonar.projectName`	Human-readable name displayed in SonarQube	Not for authorization

The following are the project names and keys to be used by the teams:

Team	Project Key	Project Name
LiL Leap	LilLeap-Project	LilLeap-Project
Leap Laugh Love	LeapLaughLove-Project	LeapLaughLove-Project
Dua Leapa	DuaLeapa-Project	DuaLeapa-Project
LeBron Market	LeMarketJames-Project	LeMarketJames-Project